

Classes e Herança

UA.DETI.POO

Herança

Herança - Conceitos

- ❖ Uma das principais características de POO em Java
- ❖ É uma forma de reutilização de Métodos e Propriedades
- ❖ Nos Interfaces
 - Herança múltipla entre interfaces
 - Reutilização da Estrutura – nomes dos métodos
- ❖ Nas classes
 - Herança simples entre classes
 - Reutilização da Estrutura e Código
 - Nomes dos métodos
 - Aceder à implementação

Classe herda/derivada de classe base

❖ A classe *Dog* herda, ou é derivada, de *Animal* quando *Animal* representa um sub-conjunto de *Animal*

❖ representa-se na forma:

```
class Dog extends Animal { /* ... */ }
```

❖ *Dog* tem acesso aos dados e métodos de *Animal*

- que não sejam privados em *Animal*

```
// Parent (Super / base ) class
class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
    void speak() {
        System.out.println("Some sound");
    }
}

// Child (derived) class: inherits from Animal
class Dog extends Animal {
    @Override
    void speak() {                // overriding parent method
        System.out.println("Woof");
    }
    void fetch() {                // new method only in Dog
        System.out.println("Dog fetches");
    }
}
```

Classe herda/derivada de classe base

❖ A classe *Dog* herda, ou é derivada, de *Animal* quando *Animal* representa um sub-conjunto de *Animal*

❖ representa-se na forma:

```
class Dog extends Animal { /* ... */ }
```

❖ *Dog* tem acesso aos dados e métodos de *Animal*

- que não sejam privados em *Animal*

Super classe



```
// Parent (Super / base ) class
class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
    void speak() {
        System.out.println("Some sound");
    }
}
```

classe



```
// Child (derived) class: inherits from Animal
class Dog extends Animal {
    @Override
    void speak() {                // overriding parent method
        System.out.println("Woof");
    }
    void fetch() {                // new method only in Dog
        System.out.println("Dog fetches");
    }
}
```

Herança simples entre classes

```
// Parent (base) class
class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
    void speak() {
        System.out.println("Some sound");
    }
}

// Child (derived) class: inherits from Animal
class Dog extends Animal {
    @Override
    void speak() {                // overriding parent method
        System.out.println("Woof");
    }
    void fetch() {                // new method only in Dog
        System.out.println("Dog fetches");
    }
}

public class ClassInheritanceDemo {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.speak();    // Woof
        d.fetch();    // Fetching...

        Animal a = new Dog(); // polymorphism
        a.speak();    // Woof
        // a.fetch(); // not allowed: Animal reference doesn't know fetch()
    }
}
```

Herança simples entre classes

```
// Parent (base) class
```

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
    void speak() {  
        System.out.println("Some sound");  
    }  
}
```

```
// Child (derived) class: inherits from Animal
```

```
class Dog extends Animal {  
    @Override  
    void speak() {                // overriding parent method  
        System.out.println("Woof");  
    }  
    void fetch() {                // new method only in Dog  
        System.out.println("Dog fetches");  
    }  
}
```

```
public class ClassInheritanceDemo {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.speak();    // Woof  
        d.fetch();    // Fetching...  
  
        Animal a = new Dog(); // polymorphism  
        a.speak();    // Woof  
        // a.fetch(); // not allowed: Animal reference doesn't know fetch()  
    }  
}
```

Relação entre Classe e super classe

Herança de Métodos

- ❖ Ao herdar métodos podemos:
 - mantê-los inalterados,
 - acrescentar-lhe funcionalidades novas ou
 - redefini-los

Herança de Métodos - herdar

```
class Person {
    private String name;
    public Person(String n) { name = n; }
    public String name() { return name; }
    public String toString() { return "PERSON";}
}
class Student extends Person {
    private int nmec;
    public Student(String s, int n) { super(s); nmec=n; }
    public int num() { return nmec; }
}
public class Test {
    public static void main(String[] args) {
        Student stu = new Student("Andreia", 55678);
        System.out.println(stu + " : " +
            stu.name() + ", " + stu.num());
    }
}
```

Herança de Métodos - redefinir

```
class Person {  
    private String name;  
    public Person(String n) { name = n; }  
    public String name() { return name; }  
    public String toString() { return "PERSON";}  
}  
  
class Student extends Person {  
    private int nmec;  
    public Student(String s, int n) { super(s); nmec=n; }  
    public int num() { return nmec; }  
    @Override  
    public String toString() { return "STUDENT"; }  
}
```

Super

- ❖ A referência **super** permite aceder às características, métodos, e construtores da “super classe” que estamos a estender.
- ❖ É uma referência à parte do objeto construída a partir da “super classe”,
- ❖ As restrições/capacidades do **super** são muita parecidas à da referência **this**.
 - this = o objecto actual
 - super = a super classe desse mesmo objecto
 - **não pode ser usada nos elementos estáticos da classe.**
- ❖ Nos construtores
 - A invocação `super(...)` tem de ser a primeira instrução

Super – invocar super classe

Aceder ao constructor
da super classe



```
class Vehicle {  
    String brand;  
    Vehicle(String brand) { this.brand = brand; }  
}  
  
class Car extends Vehicle {  
    Car(String brand) {  
        super(brand);           // call Vehicle(String)  
    }  
}
```

Aceder ao método da
super classe



```
class Animal {  
    void speak() { System.out.println("Some sound"); }  
}  
  
class Dog extends Animal {  
    @Override  
    void speak() {  
        super.speak();           // call Animal.speak()  
        System.out.println("Woof");  
    }  
}
```

Super - estender

```
class Person {  
    private String name;  
    public Person(String n) { name = n; }  
    public String name() { return name; }  
    public String toString() { return "PERSON";}  
}
```

```
class Student extends Person {  
    private int nmec;  
    public Student(String s, int n) { super(s); nmec=n; }  
    public int num() { return nmec; }  
    @Override  
    public String toString()  
    { return super.toString() + " STUDENT"; }  
}
```

Super - Construtores com parâmetros

- ❖ Em construtores com parâmetros o construtor da classe base é a primeira instrução a aparecer num construtor da classe derivada.

```
class Game {  
    int num;  
    Game(int code) { ... }  
    // ...  
}  
  
class BoardGame extends Game {  
    // ...  
    BoardGame(int code, int numPlayers) {  
        super(code);  
        // ...  
    }  
}
```

Herança e controlo de acesso

- ❖ Não podemos reduzir a visibilidade de métodos herdados numa classe derivada
 - Métodos declarados como public na classe base devem ser public nas subclasses
 - Métodos declarados como protected na classe base devem ser protected ou public nas subclasses. Não podem ser private
 - Métodos declarados sem controlo de acesso (default) não podem ser private em subclasses
 - Métodos declarados como private não são herdados

Identificação de relações entre classes

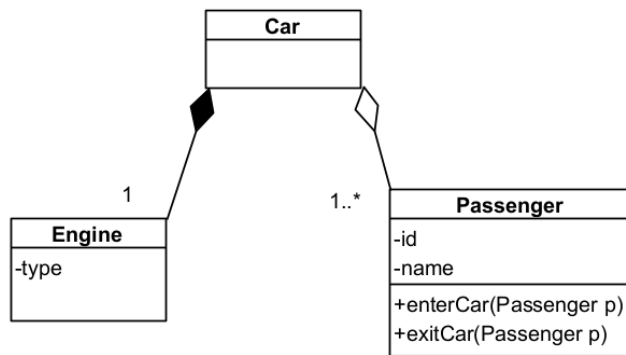
Identificação de Relações entre Classes

- ❖ Parte do processo de modelação em classes consiste em:
 - Identificar entidades candidatas a classes
 - Identificar relações entre estas entidades
- ❖ As relações entre classes identificam-se facilmente recorrendo a alguns modelos reais.
 - Por exemplo, um RelógioDigital e um RelógioAnalógico são ambos tipos de Relógio (especialização ou herança).
 - Um RelógioDigital, por seu lado, contém uma Pilha (agregação / composição).

Is-A , Has-A

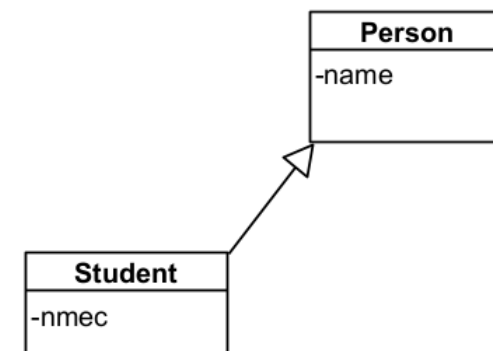
❖ Has-A

- ❖ Nova classe é baseada utilizar uma classe existente
 - has a / usa os seus recursos
 - Comportamentos
 - Propriedade



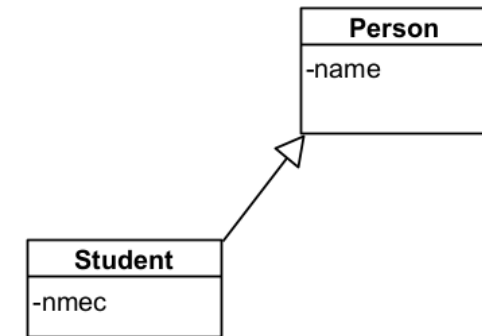
❖ Is-A

- ❖ Nova classe é baseada numa existente
 - herança
 - herdando os seus Comportamentos e Propriedade



Herança (IS-A)

- ❖ Sinais típicos de que duas classes têm um relacionamento de herança
 - Possuem aspetos comuns (dados, comportamento)
 - Possuem aspetos distintos
 - Uma é uma especialização da outra
- ❖ Exemplos:
 - Gato é um Mamífero
 - Circulo é uma Figura
 - Água é uma Bebida
 - Pinheiro é uma (IS-A) Árvore.
 - Um RelógioDigital é um (IS-A) Relógio.



Herança (IS-A)

```
public class Person {
    private String name;

    // Constructor to initialize the Person class
    public Person(String name) {
        this.name = name;
    }

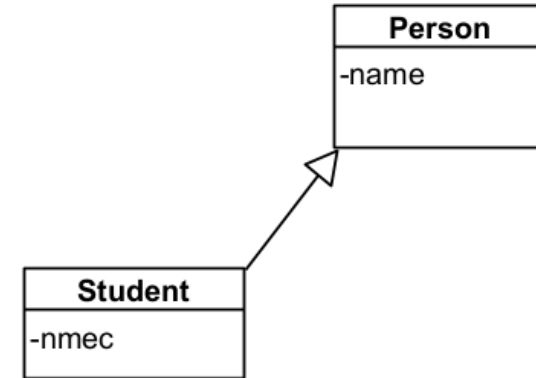
    public String getName() {        return name;    }
    public void setName(String name) {        this.name = name;    }

}

public class Student extends Person {
    private static int idCounter = 0; private int studentId;

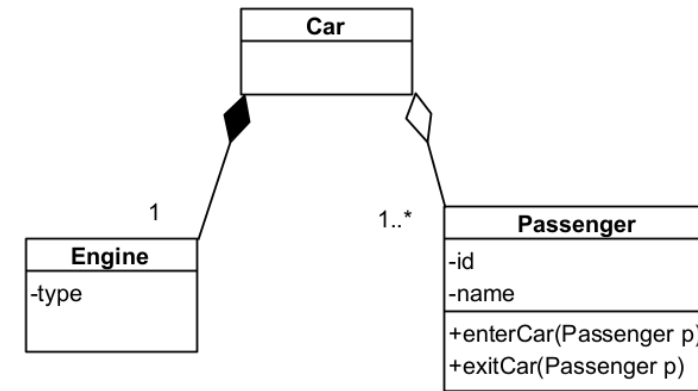
    public Student(String name) {
        super(name); // Call the superclass (Person) constructor
        this.studentId = ++idCounter; // Increment and assign the unique ID
    }
    public int getStudentId() {        return studentId;    }

}
```



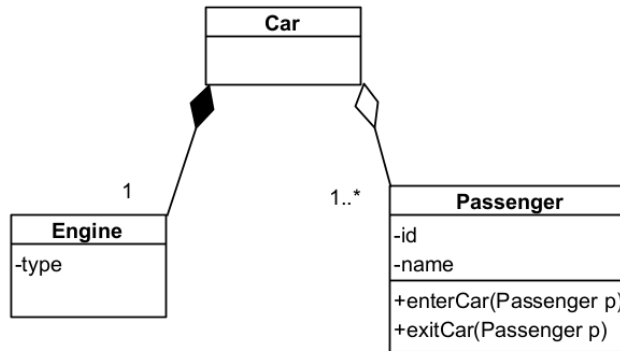
Agregação e composição (HAS-A)

- ❖ HAS-A indica que uma classe contém por objetos de outra classe.
- ❖ Composição – parte integral
 - O carro (Car) tem um motor (Engine)
 - O motor (Engine) não faz sentido fora do contexto do carro (Car)
- ❖ Agregação – pode ter / estar relacionado com
 - O carro (Car) pode ter passageiros (Passenger)
 - Os passageiros não dependem do carro para existir
- ❖ **Composição é mais forte que a agregação**



Car pode ter 1 ou mais Passenger
Car tem 1 Engine

Example: Engine e Passenger(s)



```
class Engine {
    private String type;

    public Engine(String type) {
        this.type = type;
    }

    public String getType() {
        return type;
    }
}
```

```
class Passenger {
    // Passenger properties
    private String name;
    private int id; // Unique identifier for each passenger

    public Passenger(String name, int id) {
        this.name = name;
        this.id = id;
    }

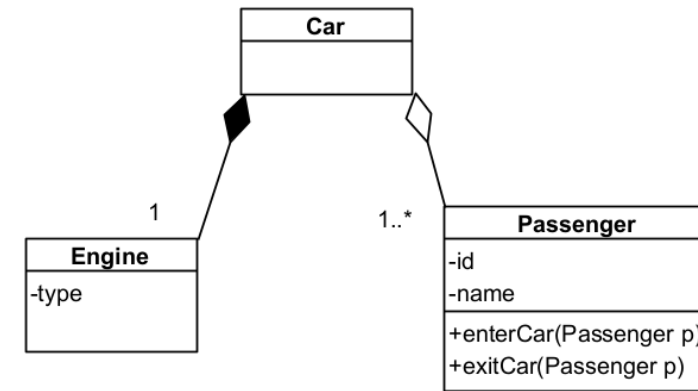
    public String getName() { return name; }
    public int getId() { return id; }

    // Override equals and hashCode to use id for equality
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Passenger passenger = (Passenger) o;
        return id == passenger.id;
    }

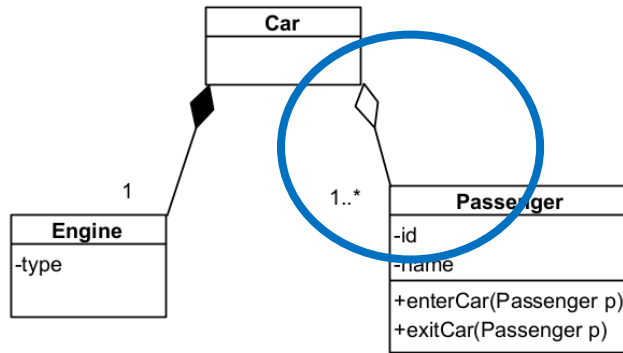
    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}
```

Agregação e composição (HAS-A)

- ❖ **HAS-A** indica que uma classe contém por objetos de outra classe.
- ❖ Composição – parte integral
 - O carro (Car) tem um motor (Engine)
 - O motor (Engine) não faz sentido fora do contexto do carro (Car)
- ❖ Agregação – pode ter / estar relacionado com
 - O carro (Car) pode ter passageiros (Passenger)
 - Os passageiros não dependem do carro para existir
- ❖ **Composição é mais forte que a agregação**



Has-A Agregação usando collections



```
class Car {

    private Engine engine;
    private List<Passenger> passengers;

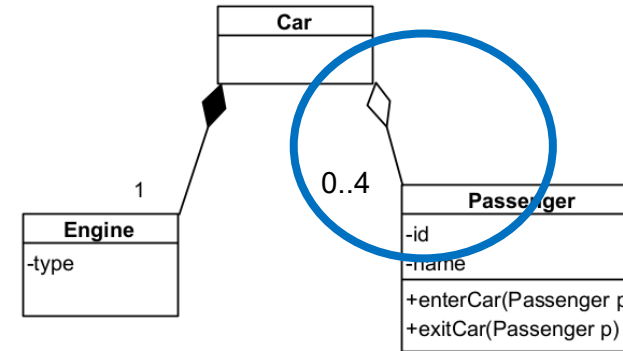
    // Car constructor
    public Car(Engine engine) {
        this.engine = engine;
        this.passengers = new ArrayList<>();
    }
    (...)
}
```

```
public void enterCar(Passenger passenger) {
    if (!passengers.contains(passenger)) {
        passengers.add(passenger);
    } else {
        System.out.println("A passenger with ID " + passenger.getId() + " is already in the car.");
    }
}

public void exitCar(int passengerId) {
    Passenger tempPassenger = new Passenger("", passengerId); // Temporary passenger to leverage equals/hashCode
    if (passengers.remove(tempPassenger)) {
        System.out.println("Passenger with ID " + passengerId + " has exited the car.");
    } else {
        System.out.println("No passenger with ID " + passengerId + " found in the car.");
    }
}
```

Has-A gerida por quem tem

```
class Car {  
    private Engine engine;  
    private List<Passenger> passengers;  
    private static final int MAX_PASSENGERS = 4;  
  
    (...)  
    // Car constructor  
    public Car(Engine engine) {  
        this.engine = engine;  
        this.passengers = new ArrayList<>();  
    }  
  
    // Method to add a passenger to the car  
    public void enterCar(Passenger passenger) {  
        if (passengers.size() >= MAX_PASSENGERS) {  
            return;  
        }  
        if (!passengers.contains(passenger)) {  
            passengers.add(passenger);  
            System.out.println(passenger.getName() + " has entered the car.");  
        } else {  
            System.out.println("A passenger is already in the car.");  
        }  
    }  
  
    (...)  
}
```



`System.out.println("The car is full.");`

Número limitado de passageiros

Has-A gerida por quem tem

```
// Main class to run the program
public class Main {
    public static void main(String[] args) {
        // Create an Engine object
        Engine myEngine = new Engine("V8");

        // Create a Car object with the Engine
        Car myCar = new Car(myEngine);

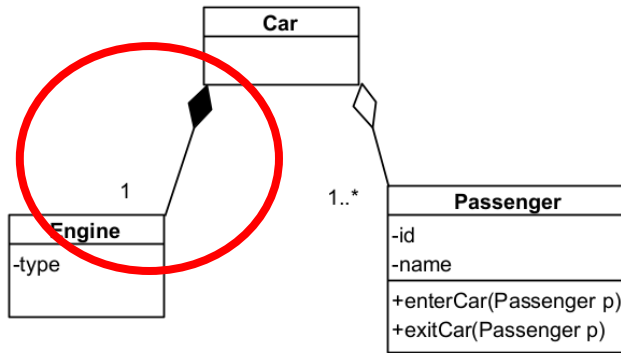
        // Add passengers to the car
        myCar.enterCar(new Passenger("Alice", 101));
        myCar.enterCar(new Passenger("Bob", 102));
        myCar.displayCarDetails();

        // Try to add Alice again with the same ID
        myCar.enterCar(new Passenger("Alice", 101));

        // Bob exits the car
        myCar.exitCar(102);

        // Display the car details after changes
        myCar.displayCarDetails();
    }
}
```

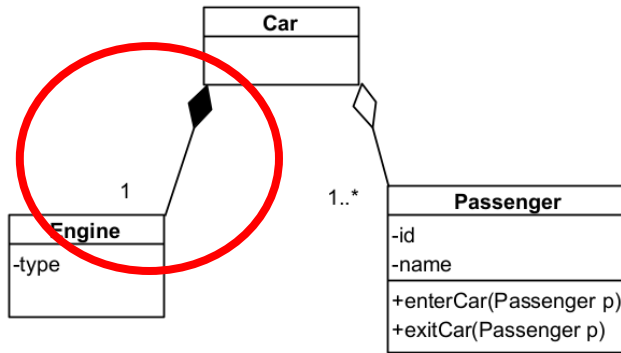
Has-A - Composição



```
class Car {
    private Engine engine;
    private List<Passenger> passengers;

    // Car constructor
    public Car(Engine engine) {
        this.engine = engine;
        this.passengers = new ArrayList<>();
    }
    (...)
}
```

Has-A - Composição



```
class Car {
    private Engine engine;
    private List<Passenger> passengers;

    // Car constructor
    public Car(Engine engine) {
        this.engine = engine;
        this.passengers = new ArrayList<>();
    }
    (...)
}
```

Neste contexto o “engine” só pode existir se estiver associado a um carro
Se o Car for destruído, o Engine também é destruído

Herança - Exemplo

```
class Person {
    private String name;
    public Person(String n) { name = n; }
    public String name() { return name; }
    public String toString() { return "PERSON";}
}
class Student extends Person {
    private int nmec;
    public Student(String s, int n) { super(s); nmec=n; }
    public int num() { return nmec; }
    public String toString() { return "STUDENT"; }
}
public class Test {
    public static void main(String[] args) {
        Person p = new Person("Joaquim");
        Student stu = new Student("Andreia", 55678);
        System.out.println(p + " : " + p.name());
        System.out.println(stu + " : " + stu.name() + ", " + stu.num());
    }
}
```

Base

Derivada

PERSON : Joaquim

STUDENT : Andreia, 55678

Herança - Exemplo

```
class Person {
    private String name;
    public Person(String n) { name = n; }
    public String name() { return name; }
    public String toString() { return "PERSON";}
}
class Student extends Person {
    private int nmec;
    public Student(String s, int n) { super(s); nmec=n; }
    public int num() { return nmec; }
    public String toString() { return "STUDENT"; }
}
public class Test {
    public static void main(String[] args) {
        Person p = new Person("Joaquim");
        Student stu = new Student("Andreia", 55678);
        System.out.println(p + " : " + p.name());
        System.out.println(stu + " : " + stu.name() + ", " + stu.num());
    }
}
```

Base

Derivada

PERSON : Joaquim

STUDENT : Andreia, 55678

Herança - Exemplo

```
class Art {
    Art() {
        System.out.println("1 Art constructor");
    }
}
class Drawing extends Art {
    Drawing() {
        System.out.println("2 Drawing constr.");
    }
}
public class Cartoon extends Drawing {
    Cartoon() {
        System.out.println("3 Cartoon constr.");
    }
    public static void main(String[] args) {
        Cartoon x = new Cartoon();
    }
}
```

1 Art constructor
2 Drawing constr.
3 Cartoon constr.

A construção é feita
a partir da classe
base

Herança - Exemplo

```
class Art {
    Art() {
        System.out.println("1 Art constructor");
    }
}
class Drawing extends Art {
    Drawing() {
        System.out.println("2 Drawing constr.");
    }
}
public class Cartoon extends Drawing {
    Cartoon() {
        System.out.println("3 Cartoon constr.");
    }
    public static void main(String[] args) {
        Cartoon x = new Cartoon();
    }
}
```

1 Art constructor

2 Drawing constr.

3 Cartoon constr.

A construção é feita a partir da classe base

Exemplos

Questões?

❖ Quais as relações entre:

- Trabalhador, Motorista, Vendedor, Administrativo e Contabilista
- Triângulo, Retângulo e Losango
- Professor, Aluno e Funcionário
- Autocarro, Viatura, Roda, Motor, Pneu, Jante

Questões?

- ❖ Represente os seguintes elementos (classes) bem como as suas relações (herança e composição)
 - Livro
 - Artigo
 - Jornal
 - Publicação
 - Autor
 - Periódico
 - Editora
 - Revista

Questões?

❖ Represente os seguintes elementos (classes) bem como as suas relações (herança e composição)

- Livro
- Artigo
- Jornal
- Publicação
- Autor
- Periódico
- Editora
- Revista

Livro

Artigo

Editora

Jornal

Publicação

Revista

Autor

Periódico

Questões?

❖ Represente os seguintes elementos (classes) bem como as suas relações (herança e composição)

- Livro
- Artigo
- Jornal
- Publicação
- Autor
- Periódico
- Editora
- Revista



Interfaces

Interfaces

- ❖ Funciona como um tipo
- ❖ Contem assinaturas de métodos que devem existir
 - implicitamente, abstratos.
 - Só podem ser *public* e/ou *abstract*.
- ❖ Contem variáveis
 - implicitamente estáticas e constantes
 - `static final` ..
- ❖ pode herdar (*implements*) mais do que uma interface.
- ❖ Uma interface pode ser vazia
 - `Cloneable`, `Serializable`

```
List<Integer> l=new ArrayList<>()

interface Desenhavel {
    public void cor(Color c);
    public void corDeFundo(Color cf);
    public void posicao(double x, double y);
    public void desenha(DrawWindow dw);
}

interface Instrument {
    // Compile-time constant:
    int i = 5; // static & final
    // Cannot have method definitions:
    void play(); // Automatically public
    String what();
    void adjust();
}

class Wind implements Instrument {
    public void play() {
        System.out.println("Wind.play()");
    }
    public String what() { return "Wind"; }
    public void adjust() { /* .. */ }
}
```


Interfaces vs Classes

❖ Não é uma classe

- não permite construtores.
- Não se pode criar uma instância da interface

❖ um protocolo para as classes que as implementam.

- implementar todos os seus métodos.
- A partir do Java 8 passou a incluir métodos *default* e *static*.

```
interface Desenhavel {  
    public void cor(Color c);  
    public void corDeFundo(Color cf);  
    public void posicao(double x, double y);  
    public void desenha(DrawWindow dw);  
}
```

```
class CirculoGrafico extends Circulo  
implements Desenhavel {  
    public void cor(Color c) {...}  
    public void corDeFundo(Color cf) {...}  
    public void posicao(double x, double y)  
{...}  
    public void desenha(DrawWindow dw) {...}  
}
```

Exemplo: List

```
import java.util.ArrayList;
import java.util.List;

public class ListInterfaceAddElements {

    public static void main(String[] args) {

        List<Integer> list = new
        list.add(1);
        list.add(3);
        list.add(4);
        // Insertion order maintained. Output: [1, 3, 4]
        System.out.println(list);

        // add 2 at index 1. Output: [1, 2, 3, 4]
        list.add(1, 2);
        System.out.println(list);

        List<Integer> list2 = new
        // append list2 to list 1
        list.addAll(list2);

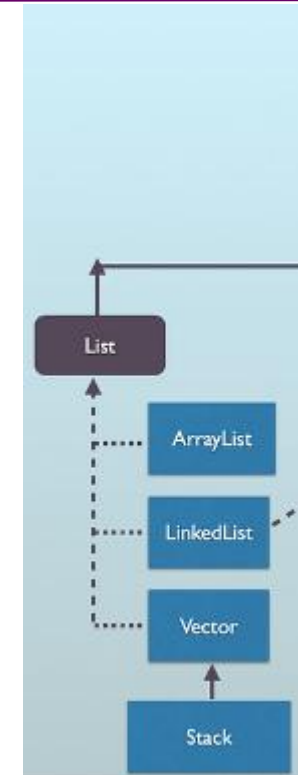
        // Order maintained and Output is: [1, 2, 3, 4, 77, 66, 55]
        System.out.println(list);

        List<Integer> list3 = new
        // adding all elements from list3 at index 3
        list.addAll(3, list3);
        // Order maintained and output is: [1, 2, 3, 44, 33, 22, 4, 77, 66, 55]
        System.out.println(list);
    }
}
```

Substituir aqui

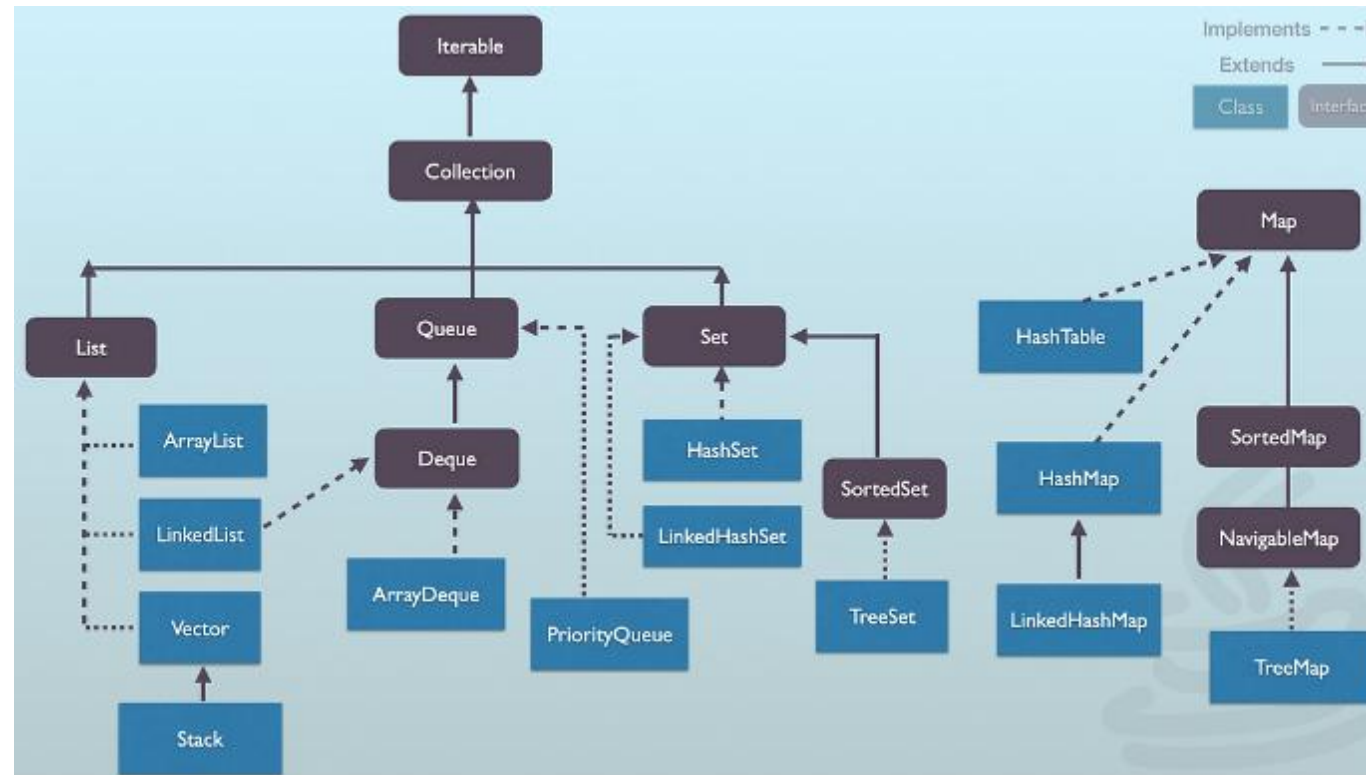
Substituir aqui

Substituir aqui



Exemplo: collections

❖ List é um interface



<https://medium.com/@rmg007/the-list-interface-in-java-cb62d524fed4>

https://www.tutorialspoint.com/java/java_list_interface.htm

List é um interface

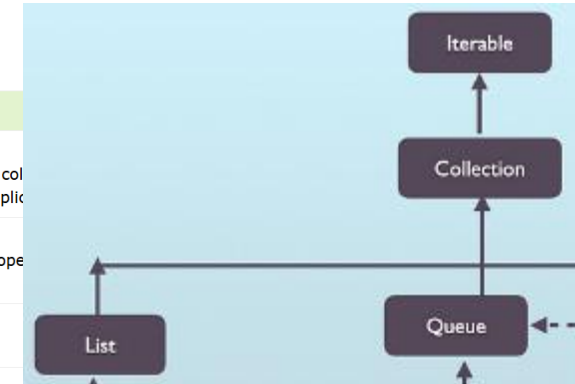
Sr.No.	Method & Description
1	void add(int index, Object obj) Inserts obj into the invoking list at the index passed in the index. Any pre-existing elements at or beyond the point of insertion are shifted up. Thus, no elements are overwritten.
2	boolean addAll(int index, Collection c) Inserts all elements of c into the invoking list at the index passed in the index. Any pre-existing elements at or beyond the point of insertion are shifted up. Thus, no elements are overwritten. Returns true if the invoking list changes and returns false otherwise.
3	Object get(int index) Returns the object stored at the specified index within the invoking collection.
4	int indexOf(Object obj) Returns the index of the first instance of obj in the invoking list. If obj is not an element of the list, -1 is returned.
5	int lastIndexOf(Object obj) Returns the index of the last instance of obj in the invoking list. If obj is not an element of the list, -1 is returned.
6	ListIterator listIterator() Returns an iterator to the start of the invoking list.
7	ListIterator listIterator(int index) Returns an iterator to the invoking list that begins at the specified index.
8	Object remove(int index) Removes the element at position index from the invoking list and returns the deleted element. The resulting list is compacted. That is, the indexes of subsequent elements are decremented by one.
9	Object set(int index, Object obj) Assigns obj to the location specified by index within the invoking list.
10	List subList(int start, int end) Returns a list that includes elements from start to end-1 in the invoking list. Elements in the returned list are also referenced by the invoking object.

List extends interface Collection

Sr.No.	Method & Description
1	void add(int index, Object obj) Inserts obj into the invoking list at the index passed in the index. Any pre-existing elements at or beyond the point of insertion are shifted up. Thus, no elements are overwritten.
2	boolean addAll(int index, Collection c) Inserts all elements of c into the invoking list at the index passed in the index. Elements at or beyond the point of insertion are shifted up. Thus, no elements are overwritten. Returns true if the invoking list changes and returns false otherwise.
3	Object get(int index) Returns the object stored at the specified index within the invoking collection.
4	int indexOf(Object obj) Returns the index of the first instance of obj in the invoking list. If obj is not found, returns -1.
5	int lastIndexOf(Object obj) Returns the index of the last instance of obj in the invoking list. If obj is not found, returns -1.
6	ListIterator listIterator() Returns an iterator to the start of the invoking list.
7	ListIterator listIterator(int index) Returns an iterator to the invoking list that begins at the specified index.
8	Object remove(int index) Removes the element at position index from the invoking list and returns the element. The resulting list is compacted. That is, the indexes of subsequent elements are adjusted.
9	Object set(int index, Object obj) Assigns obj to the location specified by index within the invoking list.
10	List subList(int start, int end) Returns a list that includes elements from start to end-1 in the invoking list. The returned list is also referenced by the invoking object.

LIST

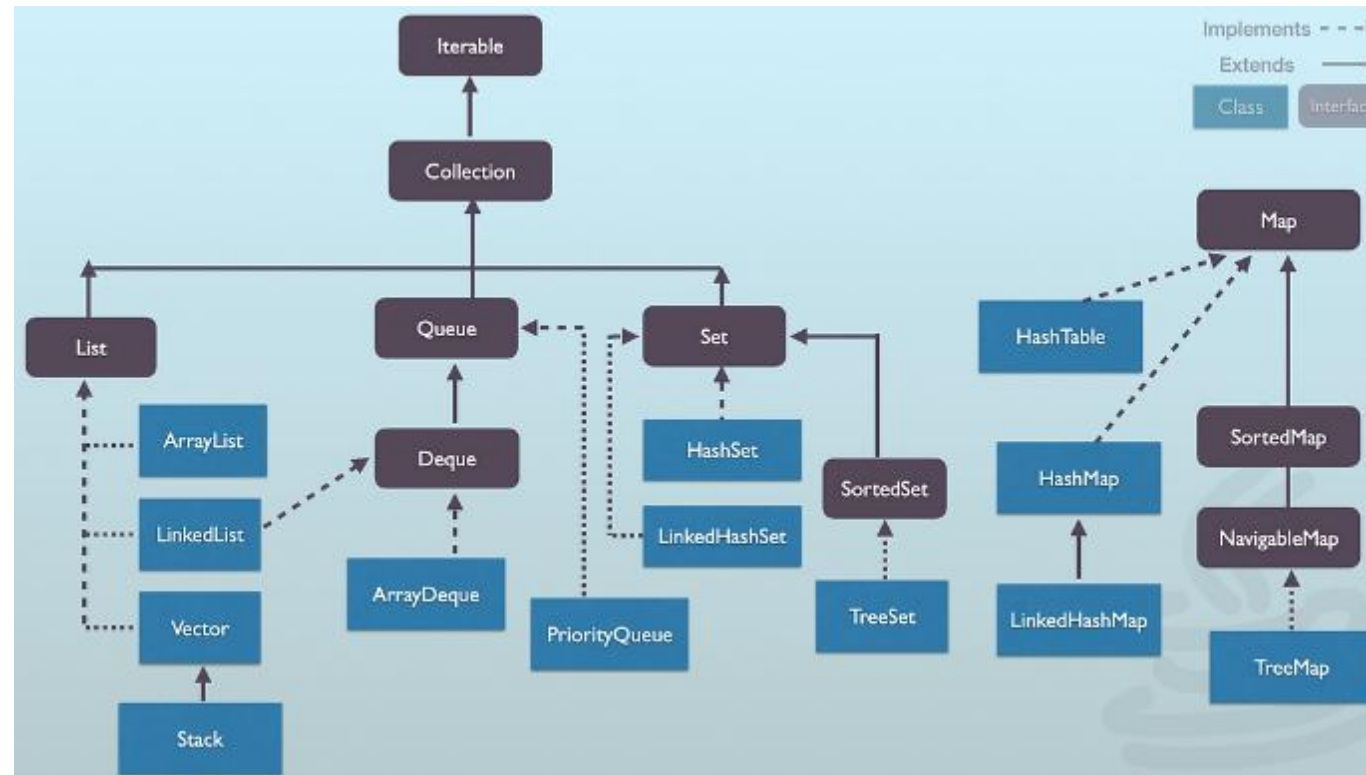
Sr.No.	Method & Description
1	boolean add(Object obj) Adds obj to the invoking collection. Returns true if obj was added to the collection, or if the collection does not allow duplicates.
2	boolean addAll(Collection c) Adds all the elements of c to the invoking collection. Returns true if the collection changes (i.e., elements were added). Otherwise, returns false.
3	void clear() Removes all elements from the invoking collection.
4	boolean contains(Object obj) Returns true if obj is an element of the invoking collection. Otherwise, returns false.
5	boolean containsAll(Collection c) Returns true if the invoking collection contains all elements of c. Otherwise, returns false.
6	boolean equals(Object obj) Returns true if the invoking collection and obj are equal. Otherwise, returns false.
7	int hashCode() Returns the hash code for the invoking collection.
8	boolean isEmpty() Returns true if the invoking collection is empty. Otherwise, returns false.
9	Iterator iterator() Returns an iterator for the invoking collection.
10	boolean remove(Object obj) Removes one instance of obj from the invoking collection. Returns true if the element was removed. Otherwise, returns false.
11	boolean removeAll(Collection c) Removes all elements of c from the invoking collection. Returns true if the collection changed (i.e., elements were removed). Otherwise, returns false.
12	boolean retainAll(Collection c) Removes all elements from the invoking collection except those in c. Returns true if the collection changed (i.e., elements were removed). Otherwise, returns false.
13	int size() Returns the number of elements held in the invoking collection.
14	Object[] toArray() Returns an array that contains all the elements stored in the invoking collection. The array elements are copies of the collection elements.



COLLECTION

Exemplo: collections

❖ List é um interface



<https://medium.com/@rmg007/the-list-interface-in-java-cb62d524fed4>

https://www.tutorialspoint.com/java/java_list_interface.htm

Exemplo: ArrayList as List

```
import java.util.ArrayList;
import java.util.List;

public class ListInterfaceAddElements {

    public static void main(String[] args) {

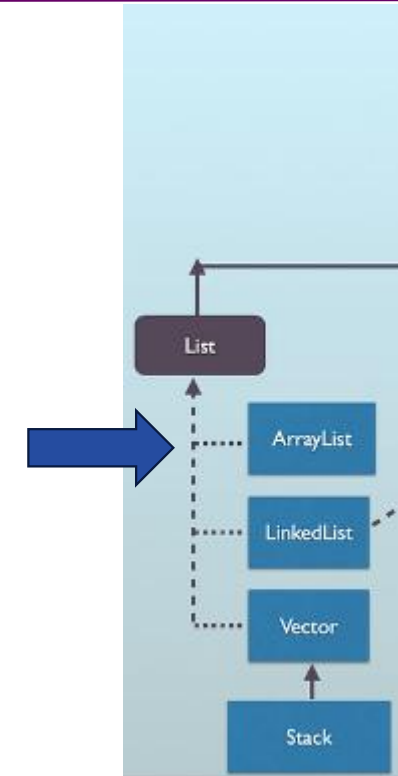
        List<Integer> list = new ArrayList<>();
        list.add(1);
        list.add(3);
        list.add(4);
        // Insertion order maintained. Output: [1, 3, 4]
        System.out.println(list);

        // add 2 at index 1. Output: [1, 2, 3, 4]
        list.add(1, 2);
        System.out.println(list);

        List<Integer> list2 = new ArrayList<>(List.of(77, 66, 55));
        // append list2 to list 1
        list.addAll(list2);

        // Order maintained and Output is: [1, 2, 3, 4, 77, 66, 55]
        System.out.println(list);

        List<Integer> list3 = new ArrayList<>(List.of(44, 33, 22));
        // adding all elements from list3 at index 3
        list.addAll(3, list3);
        // Order maintained and output is: [1, 2, 3, 44, 33, 22, 4, 77, 66, 55]
        System.out.println(list);
    }
}
```



Interfaces - Exemplos

```
interface Instrument {  
    // Compile-time constant:  
    int i = 5; // static & final  
    // Cannot have method definitions:  
    void play(); // Automatically public  
    String what();  
    void adjust();  
}  
  
class Wind implements Instrument {  
    public void play() {  
        System.out.println("Wind.play()");  
    }  
    public String what() { return "Wind"; }  
    public void adjust() { /* .. */ }  
}
```


Interfaces a partir de Java 8

❖ Default methods

- Podemos definir o corpo dos métodos na interface

❖ Static methods

- Podemos definir o corpo de métodos estáticos na interface. Devem ser invocados sobre a interface (Métodos de Interface)

❖ Functional interfaces

- *(vamos falar nisto mais tarde...)*

Key Differences vs Static Methods in Interfaces

Feature	default method	static method
Called from	Object instance	Interface name
Inherited by implementing class	✓ Yes	✗ No
Can be overridden	✓ Yes	✗ No

Example calls:

Java

```
p.sayHello();           // default method
Greeting.log();         // static method
```

Interfaces a partir de Java 8

❖ Default Methods

- Oferecem uma **implementação por omissão**
- Podem ser reescritos nas classes que implementam a interface

```
public interface InterfaceOne {  
    default void defMeth() { //... do something  
    }  
}  
  
public class MyClass implements InterfaceOne {  
    @Override  
    public void defMeth() { // ... do something  
    }  
}
```

Default methods

```
interface Greeting {  
    // default method  
    default void sayHello() {  
        System.out.println("Hello from the interface!");  
    }  
}  
  
class Person implements Greeting {  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Person p = new Person();  
        p.sayHello(); // calling the inherited default method  
    }  
}
```

Hello from the interface!

```
interface Greeting {  
    default void sayHello() {  
        System.out.println("Hello from the interface!");  
    }  
}  
  
class Person implements Greeting {  
    @Override  
    public void sayHello() {  
        System.out.println("Hello from the Person class!");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Person p = new Person();  
        p.sayHello();  
    }  
}
```

Hello from the Person class!

Interfaces a partir de Java 8

❖ Static Methods

- Similares aos default methods
- Não **podem ser reescritos nas classes que implementam a interface**
- Só podem ser invocados através da interface onde estão definidos (não através das classes que implementam a interface)

```
public interface Interface2 {  
    static void stMeth() { //... do  
        something  
    }  
}
```

```
public class MyClass implements  
Interface2 {  
    @Override  
    public void stMeth() { // ...  
do something  
    }  
}
```



Static methods in interfaces

```
interface MathUtils {  
    // static method in interface  
    static int add(int a, int b) {  
        return a + b;  
    }  
}  
  
public class Main {  
    public static void main(String[] args)  
    {  
        // calling the static method using the interface name  
        int result = MathUtils.add(5, 3);  
  
        System.out.println("Result: " + result);  
    }  
}
```

```
interface Logger {  
    static void log(String message) {  
        System.out.println("LOG: " + message);  
    }  
}  
  
class App implements Logger {  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Logger.log("Application started");  
    }  
}
```

Interfaces a partir de Java 8

❖ Default methods

- Podemos definir o corpo dos métodos na interface

❖ Static methods

- Podemos definir o corpo de métodos estáticos na interface. Devem ser invocados sobre a interface (Métodos de Interface)

❖ Functional interfaces

- *(vamos falar nisto mais tarde...)*



A RETORNAR MAIS
À FRENTE NO
CURSO

Pode aparecer tudo junto...

```
// Interface
interface Drivable {
    void drive();
}

// Base class
class Vehicle {
    protected String brand;

    Vehicle(String brand) {
        this.brand = brand;
    }

    void honk() {
        System.out.println(brand + ": beep!");
    }
}
```

```
// Subclass that extends a class AND implements an interface
class Car extends Vehicle implements Drivable {
    Car(String brand) {
        super(brand);
    }

    @Override
    public void drive() {
        System.out.println(brand + ": driving");
    }
}

public class BothInheritanceDemo {
    public static void main(String[] args) {
        Car car = new Car("Toyota");
        car.honk();    // from Vehicle
        car.drive();   // from Drivable

        Vehicle v = car;    // use as base class
        v.honk();

        Drivable d = car;    // use as interface
        d.drive();
    }
}
```

Pode aparecer tudo junto...

```
// Interface
interface Drivable {
    void drive();
}

// Base class
class Vehicle {
    protected String brand;

    Vehicle(String brand) {
        this.brand = brand;
    }

    void honk() {
        System.out.println(brand + ": beep!");
    }
}
```

```
// Subclass that extends a class AND implements an interface
class Car extends Vehicle implements Drivable {
    Car(String brand) {
        super(brand);
    }

    @Override
    public void drive() {
        System.out.println(brand + ": driving");
    }
}

public class BothInheritanceDemo {
    public static void main(String[] args) {
        Car car = new Car("Toyota");
        car.honk();    // from Vehicle
        car.drive();   // from Drivable

        Vehicle v = car;    // use as base class
        v.honk();

        Drivable d = car;    // use as interface
        d.drive();
    }
}
```


Herança

```
class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
}

interface Flyable {
    void fly();
}

class Bird extends Animal implements Flyable {

    public void fly() {
        System.out.println("Bird flies");
    }
}

public class Main {
    public static void main(String[] args) {
        Bird b = new Bird();
        b.eat();
        b.fly();
    }
}
```

```
interface Flyable {
    void fly();
}

interface Swimmable {
    void swim();
}

class Duck implements Flyable, Swimmable {

    public void fly() {
        System.out.println("Duck flies");
    }

    public void swim() {
        System.out.println("Duck swims");
    }
}

public class Main {
    public static void main(String[] args) {
        Duck d = new Duck();
        d.fly();
        d.swim();
    }
}
```

Herança

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}
```

```
interface Flyable {  
    void fly();  
}
```

```
class Bird extends Animal implements Flyable {  
    public void fly() {  
        System.out.println("Bird flies");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Bird b = new Bird();  
        b.eat();  
        b.fly();  
    }  
}
```

Herança simples
entre classes



```
interface Flyable {  
    void fly();  
}
```

```
interface Swimmable {  
    void swim();  
}
```

```
class Duck implements Flyable, Swimmable {
```

```
    public void fly() {  
        System.out.println("Duck flies");  
    }
```

```
    public void swim() {  
        System.out.println("Duck swims");  
    }
```

```
public class Main {  
    public static void main(String[] args) {  
        Duck d = new Duck();  
        d.fly();  
        d.swim();  
    }  
}
```

Herança múltipla
entre interfaces



Classes partilham interfaces (foco na forma)

```
interface Drivable {
    void drive(); // abstract method
}

class Car implements Drivable {
    @Override
    public void drive() {
        System.out.println("Car driving");
    }
}

class Bike implements Drivable {
    @Override
    public void drive() {
        System.out.println("Bike riding");
    }
}
```

```
public class InterfaceInheritanceDemo {
    public static void main(String[] args) {
        Drivable d1 = new Car();
        Drivable d2 = new Bike();

        d1.drive(); // Car driving
        d2.drive(); // Bike riding
    }
}
```



Não preciso saber a classe, apenas que é “Drivable”

Sumário - Porquê herança?

- ❖ Muitos objetos reais apresentam esta característica
- ❖ Permite criar classes mais simples com funcionalidades mais estanques e melhor definidas
 - Devemos evitar classes com interfaces muito "extensas"
- ❖ Permite reutilizar e estender interfaces e código
- ❖ Permite tirar partido do polimorfismo (próxima aula)

Herança - Boas Práticas

- ❖ **Programar para a interface e não para a implementação**
 - Muitas vezes herança entre interfaces basta e é recomendável
- ❖ Procurar aspetos comuns a várias classes e promovê-los a uma classe base
- ❖ Minimizar os relacionamentos entre objetos e organizar as classes relacionadas dentro de um mesmo package
- ❖ Usar herança criteriosamente – sempre que possível favorecer a composição

Uma paragem: final

Final

- ❖ O classificador final indica "não pode ser mudado"
- ❖ A sua utilização pode ser feita sobre:
 - Dados - constantes
 - Métodos - não redefiníveis
 - Classes - não herdadas
 - Fixar referencias a objectos

Usage

`final variable`

`final method`

`final class`

Case

`final variable`

`final method`

`final class`

`final reference`

Allowed

assign once

inherit

create objects

modify object state

Not Allowed

reassign value

override

extend class

change reference

Final – dados constantes

❖ A **final variable** cannot be reassigned after it is initialized.

❖ Key Points

❖ Value must be **assigned once**

❖ Cannot be changed later

❖ Often written in **UPPERCASE** by convention

```
public class Main {  
    public static void main(String[] args) {  
  
        final double PI = 3.14159;  
  
        final int x = 10;  
  
        // x = 20; // ✗ compile-time error  
  
        System.out.println(x);  
    }  
}
```


Final – métodos não redefiníveis

- ❖ A **final method** cannot be overridden by subclasses.
- ❖ **Meaning**
- ❖ Subclasses **inherit the method**
- ❖ But **cannot override it**

```
class Animal {  
    final void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
  
    // ✗ This would cause an error  
    // void sound() {  
    //     System.out.println("Dog barks");  
    // }  
}
```

Final classes

- ❖ A **final class cannot be extended** (no inheritance).

```
final class Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}
```

```
// ❌ Not allowed  
// class Dog extends Animal {}
```

Final – reference to objects

- ❖ A **final reference cannot point to another object**, but the object's **internal state can still change**.

```
class Person {  
    String name;  
}  
  
public class Main {  
    public static void main(String[] args) {  
        final Person p = new Person();  
        p.name = "Alice";    // allowed  
  
        // p = new Person();  ✗ not allowed  
    }  
}
```

A combined example

```
// ❶ final class → cannot be extended
final class Utility {
    void printMessage() {System.out.println("Utility class method"); }
}

// Parent class
class Animal {
    // ❷ final method → cannot be overridden by subclasses
    final void sleep() {    System.out.println("Animal is sleeping");}

    void eat() {    System.out.println("Animal eats");}
}

class Dog extends Animal {
    // ✗ This would cause a compile-time error
    // because sleep() is final in Animal
    /*    void sleep() {System.out.println("Dog sleeping");    }    */

    void bark() {    System.out.println("Dog barks");    }
}

// Class with constants
class Constants {
    // ❸ public static final → constant value
    public static final double PI = 3.14159;
}

class Person {
    String name;
}
```

```
public class Main {
    public static void main(String[] args) {

        // ❹ final variable → cannot be reassigned
        final int x = 10;
        System.out.println("Final variable x = " + x);

        // ✗ Not allowed
        // x = 20;

        // Using constant
        System.out.println("PI = " + Constants.PI);

        // Using inherited final method
        Dog d = new Dog();
        d.eat();
        d.sleep(); // inherited final method
        d.bark();

        // ❺ final reference
        final Person p = new Person();

        // Allowed: modify object state
        p.name = "Alice";
        System.out.println("Person name: " + p.name);

        // ✗ Not allowed: change reference
        // p = new Person();

        // Using final class
        Utility u = new Utility();
        u.printMessage();

        // ✗ Not allowed: cannot extend Utility class
        // class Test extends Utility {}
    }
}
```

give some example to illustrate all for of do and don't over final in java
all combined in some code sample with comments

Real world: Public static final ...

- ❖ A very common pattern in Java is using **public static final** to define **constants**.
- ❖ **public** → accessible from anywhere
- ❖ **static** → belongs to the class (no object needed)
- ❖ **final** → value cannot change

```
class Constants {  
  
    public static final double PI = 3.14159;  
    public static final int MAX_USERS = 100;  
    public static final String APP_NAME = "MyApplication";  
  
}  
  
public class Main {  
    public static void main(String[] args) {  
  
        System.out.println(Constants.PI);  
        System.out.println(Constants.APP_NAME);  
  
        int users = Constants.MAX_USERS;  
        System.out.println("Max users: " + users);  
    }  
}
```

The END